

naissance

Audit date - 2026-05-06 18:24

Angular ^21.2.0 - 8 TS files - 911 LOC

54/100

Grade D

Faible - problemes significatifs, refactoring urgent conseille.

9 issues detected

0

CRITICAL

8

IMPORTANT

1

MINOR

Project overview

Angular version	^21.2.0
TypeScript files	8
HTML templates	2
Components / Services / Modules	2 / 1 / 0
Pipes / Guards	0 / 0
Total lines of code	911
Tests detected	No

Severity summary



Critical: 0 Important: 8 Minor: 1

Issues detail

Type Safety

IMPORTANT

TYPE001 Usage de 'any' TypeScript (3)

Le type ``any`` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

Fix: Typer explicitement (interface, type, générique). Utiliser ``unknown`` si le type est vraiment inconnu.

OCCURRENCES

```
src/app/services/niambay.service.ts:19
private recognition: any = null;

src/app/services/niambay.service.ts:97
} catch (error: any) {

src/app/services/niambay.service.ts:175
this.recognition.onresult = (event: any) => {
```

IMPORTANT

TYPE002 Cast 'as any' explicite (1)

Un cast ``as any`` désactive volontairement TypeScript pour cette expression. Different de ``: any`` (TYPE001) qui declare un type ambigu : ``as any`` est un acte explicite de bypass du verificateur. Apparaît souvent quand un dev se bat avec un type de librairie tiers, ou quand un payload API renvoie une structure non typée. Le probleme : le compilateur ne peut plus garantir que les acces suivants (``.foo``, ``.bar()``) sont valides - un refactor de la source ne mettra plus a jour les usages, et un `null/undefined` dans le payload ne sera pas detecte au build.

Fix: 1) Si la forme est connue : declarer une ``interface`` ou ``type`` et caster vers ce type (``as User``). 2) Si la forme est partiellement connue : ``as Partial<User>`` ou ``as Pick<User, 'id'|'name'>``. 3) Si la forme est vraiment inconnue : caster vers ``unknown`` puis valider avec un type guard avant l'acces (``if (typeof x === 'object' && x !== null && 'id' in x)``). 4) Pour les reponses HTTP : utiliser ``http.get<MyType>(url)`` ou un schema runtime (zod, yup) pour valider la forme avant cast. Le cast ``as unknown`` est preferable a ``as any`` car il force au moins une etape de validation explicite.

OCCURRENCES

```
src/app/services/niambay.service.ts:159
const SpeechRecognition = (window as any).SpeechRecognition || (window as any).webkitSpeechReco...
```

Architecture



ARCH002 URL hardcodée dans le code (1)

Une URL hardcodée empêche de switcher entre dev/staging/prod sans rebuild. Force un commit pour changer un endpoint. Mauvaise pratique multi-environnements.

Fix: Déplacer l'URL dans `src/environments/environment.ts` et `environment.prod.ts`. Utiliser `environment.apiUrl` dans le code.

OCCURRENCES

[src/app/services/niambay.service.ts:125](#)

```
const response = await fetch('https://api.anthropic.com/v1/messages', {
```

Memory Leaks

IMPORTANT JS001 setTimeout/setInterval sans cleanup (1)

Un `setTimeout` ou surtout `setInterval` lance dans un composant qui n'est jamais clear continue a tourner apres la destruction du composant. Sur une SPA Angular, accumuler des intervalles oublies = memory leak progressif + appels reseau fantomes. Different de RxJS subscriptions (gere par MEM001).

Fix: Garder la reference (`this.timerId = setTimeout(...)`) et appeler `clearTimeout(this.timerId)` dans `ngOnDestroy()`. Ou mieux : utiliser `interval(N).pipe(takeUntilDestroyed())` (Angular 16+) qui s'auto-nettoie.

OCCURRENCES

[src/app/services/niambay.service.ts:108](#)

```
setTimeout(() => this.state.set('idle'), 3000);
```

Performance

IMPORTANT PERF002 Route sans lazy loading (2)

Les routes chargées eagerly augmentent le bundle initial et ralentissent le démarrage.

Fix: Remplacer `component: MyComponent` par `loadComponent: () => import('./my.component').then(m => m.MyComponent)`

OCCURRENCES

[src/app/app.routes.ts:6](#)

```
{ path: 'panel', component: PanelComponent },
```

[src/app/app.routes.ts:7](#)

```
{ path: '**', component: CircleComponent },
```

Code Quality

MINOR DEBUG001 console.log en production (1)

Les console.log oubliés exposent des données internes en prod et polluent la console.

Fix: Supprimer ou remplacer par un service de logging. Configurer `build.optimization.scripts` pour stripper en prod.

OCCURRENCES

[src/main.ts:5](#)

```
bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

Lazy loading

Eager routes (no lazy)

2

Lazy routes	0
Lazy loading ratio	0%

Less than 50% of routes use lazy loading. Each eager route adds to the initial bundle. Migrate to `loadComponent` (Angular 15+) for the heaviest routes first.

Refactoring plan

This month - Important

- Usage de 'any' TypeScript (TYPE001)

Le type ``any`` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

- URL hardcodée dans le code (ARCH002)

Une URL hardcodée empêche de switcher entre dev/staging/prod sans rebuild. Force un commit pour changer un endpoint. Mauvaise...

- `setTimeout/setInterval` sans cleanup (JS001)

Un ``setTimeout`` ou surtout ``setInterval`` lance dans un composant qui n'est jamais clear continue à tourner après la destruction...

- Cast 'as any' explicite (TYPE002)

Un cast ``as any`` désactive volontairement TypeScript pour cette expression. Différent de ``: any`` (TYPE001) qui déclare un type ...

- Route sans lazy loading (PERF002)

Les routes chargées eagerly augmentent le bundle initial et ralentissent le démarrage.

On the roadmap - Minor

- `console.log` en production (DEBUG001)

Les `console.log` oubliés exposent des données internes en prod et polluent la console.

This report is produced by automated static analysis. It does not replace a thorough manual review by an experienced Angular developer.